

Phase 2 日報機能 要件定義書 (MVP)

現場 OS Lite

作成日：2026 年 6 月 21 日 / 方針：MVP 最優先・建設会社ヒアリングで検証できる最小構成

1. 目的とスコープ

目的

現場ごとに「その日の作業内容」をスマホで記録・閲覧できるようにする。建設会社へのヒアリングで「日報をスマホで付ける」体験を見せ、項目・運用の妥当性を検証することがゴール。

MVP の考え方

- 既存の Phase 1（認証・会社・現場）の上に、最小の「日報 CRUD」を載せるだけ。
- 凝った機能（承認・PDF・集計）は入れない。まず「書ける・見れる・直せる」を最短で。
- データは既存同様 company_id + RLS で会社単位に隔離。日報は現場（site）に紐づく。

やること / やらないこと

やること (MVP)	やらないこと (後続フェーズ)
日報の 作成 / 一覧 / 詳細 / 編集	承認・ワークフロー (→ Phase 7)
現場ごとの日報記録・閲覧	PDF / 報告書出力 (→ Phase 6)
天候・作業人数などの基本項目	日報への写真添付 (→ Phase 4 と統合)
自社のみ閲覧 (RLS)	集計・分析・グラフ
作成者の記録 (created_by)	テンプレ / 音声入力 / オフライン同期

2. ユーザーストーリー (MVP)

- 監督として、現場を選んで今日の作業内容を記録したい（天候・人数も）。
- 過去の日報を新しい順に一覧で振り返りたい。
- 1 件の日報を詳細表示したい。
- 書き間違いを後から編集したい。
- 自社の日報だけが見え、他社の日報は見えない。

3. 画面一覧

#	画面	主な要素	遷移
S1	日報一覧（現場別）	その現場の日報を新しい順に表示（作業日・天候・作業内容の冒頭）。空/エラー・pull-to-refresh。作成 FAB	現場詳細→S1 / 行→S3 / +→S2
S2	日報作成	作業日（既定=今日）・天候（選択）・作業内容（必須）・作業人数（任意）。保存（ローディング/エラー）	保存成功→S1
S3	日報詳細	作業日・現場名・天候・作業内容・作業人数・作成者・更新日時。編集ボタン	編集→S4
S4	日報編集	S2 と同じフォーム（既存値プリフィル）。更新ボタン	更新成功→S3

導線：既存「現場詳細」に「日報」ボタンを追加し S1 へ。S2/S4 は同一フォームを共有（新規/編集をモード切替）。*任意（MVP 外）：Home から全現場の日報フィード。MVP は現場別のみで十分。

画面遷移

現場詳細(既存) —[日報]→ S1 日報一覧 —[+]→ S2 日報作成 —保存→ S1
—[行タップ]→ S3 日報詳細 —[編集]→ S4 日報編集 —更新→ S3

4. DB 設計

テーブル: reports

カラム	型	制約 / 既定	説明
id	uuid	PK, gen_random_uuid()	日報 ID
company_id	uuid	NOT NULL, 既定 current_company_id()	会社 (RLS・自動付与)
site_id	uuid	NOT NULL, FK→sites	対象現場
report_date	date	NOT NULL, 既定 current_date	作業日
weather	text	NULL 可	天候(sunny/cloudy/rainy/snowy)
work_content	text	NOT NULL	作業内容 (本文・必須)
worker_count	int	NULL 可, CHECK ≥ 0	作業人数
created_by	uuid	既定 auth.uid(), FK→auth.users	作成者
created_at	timestampz	NOT NULL, now()	作成日時
updated_at	timestampz	NOT NULL, now() (トリガ更新)	更新日時

- ・ インデックス: (site_id, report_date desc) (現場別・日付順)
- ・ updated_at は BEFORE UPDATE トリガで自動更新

RLS (自社のみ)

company_id = current_company_id() を select / insert / update / delete に適用 (Phase 1.2 sites と同方針)。
insert は既定値+ with check で自社固定。**拡張ポイント**: 編集を「作成者のみ」に絞る場合は update を created_by = auth.uid() に変更。

マイグレーション骨子 (supabase/migrations/0004_reports.sql 想定)

```
create table if not exists public.reports (  
  id uuid primary key default gen_random_uuid(),  
  company_id uuid not null default public.current_company_id()  
    references public.companies(id) on delete cascade,  
  site_id uuid not null references public.sites(id) on delete cascade,  
  report_date date not null default current_date,  
  weather text,  
  work_content text not null,  
  worker_count int check (worker_count >= 0),  
  created_by uuid default auth.uid() references auth.users(id),  
  created_at timestampz not null default now(),  
  updated_at timestampz not null default now()  
);  
create index if not exists reports_site_date_idx  
  on public.reports(site_id, report_date desc);  
alter table public.reports enable row level security;  
-- select/insert/update/delete: company_id = current_company_id()  
-- updated_at 自動更新トリガ
```

※ 適用は dev / prod の SQL Editor で実行 (Phase 1 と同じ運用)。

5. 実装方針 (既存パターン踏襲)

- ・ **feature-first**: lib/features/reports/{data, application, presentation}
- ・ Report モデル (freezed+json、snake_case、天候ラベル helper)
- ・ ReportRepository (抽象 interface + Supabase 実装) : listBySite / fetch / create / update / delete
- ・ Provider: reportsBySiteProvider.family(siteId) / reportDetailProvider.family(id)
- ・ Controller: ReportFormController (AsyncNotifier、create/update、成功時に一覧 invalidate)
- ・ テスト: モデル fromJson / FormController (fake repo) / 一覧画面 (fake repo)

6. 工数見積もり

#	タスク	人日	備考
1	DB マイグレーション (reports+RLS+トリガ)	0.25	既存流用
2	Report モデル(freezed)+天候 helper	0.25	
3	ReportRepository (CRUD)	0.5	

#	タスク	人日	備考
4	Provider + Form(作成/編集)Controller	0.5	
5	日報一覧画面(S1)	0.5	sites 一覧流用
6	作成/編集フォーム(S2/S4 共有)	0.75	日付/天候選択含む
7	日報詳細画面(S3)	0.25	
8	現場詳細への導線+ルーティング	0.25	
9	テスト (model/controller/list)	0.5	
10	analyze/test/ビルド/実機検証	0.5	
11	ドキュメント・SQL 適用ガイド	0.25	
—	合計	約 4.5	純粋な開発工数の目安

本プロジェクト (AI 実装) での実時間目安	
🤖 AI 実装 (コード一式+テスト+analyze/test 緑+push)	約 40～60 分
🧑 あなたの作業 (dev に 0004 SQL 適用 + 実機検証)	約 15 分
合計 (壁時計)	約 1 時間

7. Phase 2 完了条件

#	完了条件	確認方法
1	日報作成成功	現場を選び入力→保存→一覧に出る
2	日報一覧表示成功	現場別に新しい順で表示
3	日報詳細表示成功	1 件タップで全項目表示
4	日報編集成功	既存値を直して更新→詳細反映
5	RLS 確認	company_id 自動付与・自社のみ取得
6	analyze 成功	flutter analyze → No issues
7	test 成功	flutter test → 全件パス
8	dev 実機検証	iOS シミュレータで 1～5 を目視確認

本番化前の運用タスク（任意・コード変更不要）：prod へ 0004_reports.sql を適用。

8. 建設会社ヒアリングで確認したいこと（検証観点）

MVP を見せながら、以下を聞いて次の優先度を定める：

1. **入力項目は過不足ないか？**（天候・作業人数は要る？ 翌日予定・使用機材・安全確認は要る？）
2. **写真は日報に紐づけたいか？**（現場単位の今の写真で足りるか／日報ごとに付けたいか → Phase 4 と統合判断）
3. **承認・確認フローは要るか？**（上長が承認する運用か → Phase 7）
4. **PDF/紙の体裁で出したいか？**（提出先がある → Phase 6 優先度 UP）
5. **だれが書くか／編集権限**（職人本人 / 監督のみ / 作成者だけ編集 → RLS 方針）
6. **1 日 1 現場 1 枚か、複数枚か**（運用粒度）

この回答で、Phase 2 の項目追加や Phase 3～7 の優先順位を調整する。

関連: docs/ROADMAP.md（全体ロードマップ）／踏襲元 = Phase 1.2 現場一覧。